山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验 10 堆及其应用		
实验学时：2	实验日期： 2023.11.22	

实验目的：

实现堆以及堆的应用

软件开发工具：

DEV C++

1. 实验内容

A ： 堆的操作

创建 最小堆类。最小堆的存储结构使用 数组。提供操作:插入、删除、初始化。题目第一个操作是建堆操作，接下来是对堆的插入和删除操作，插入和删除都在建好的堆上操作。

B ： 霍夫曼编码

输入一串小写字母组成的字符串（不超过 1000000)，输出这个字符串通过 Huffman 编码后的长度。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

A ： 堆的操作：

数据结构与算法：最小堆，完全二叉树结构

整体思路描述：

（1）初始化堆：将堆的初始元素输入到数组中，找到最后一个有孩子的节点，对以该节点为根的树进行堆化，依次向上遍历，直到遍历到序号为 1 的节点，整棵树完成堆化，实现了堆的初始化。

（2）插入操作：将新节点与父节点 value 值比较，如果节点没有到堆顶且父节点比现在节点值大，则将父节点下沉，新节点的位置索引更新为父节点索引。

（3）删除操作：记录最后一个节点值。记录堆顶元素的孩子中最小的那一个，将其与所记录的最后一个节点值相比较。如果最后一个节点比孩子中的最小者要大，则将最小的孩子上移动，记录该孩子的两个孩子中最小的那一个再与记录的最后一个节点值相比较。直到最后一个节点值比两个孩子都要小，则停止循环，将最后一个节点插入到停止时的父节点处。

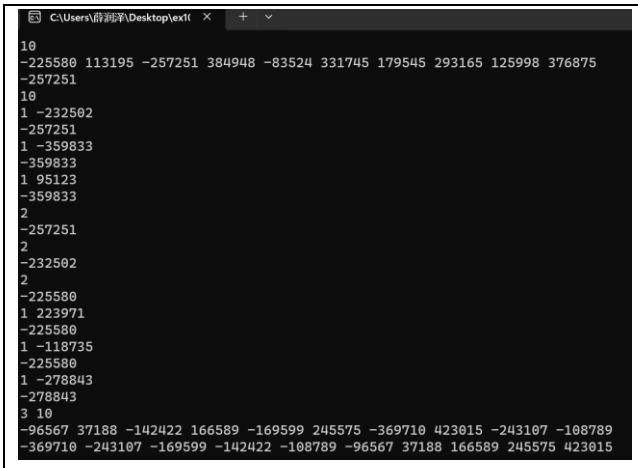
B：霍夫曼编码

数据结构与算法：最小堆

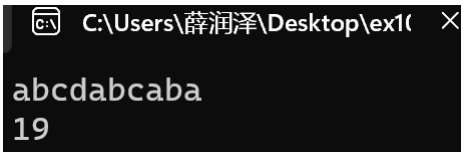
思路描述：用最小堆来存储霍夫曼树，最开始是存储最多 26 个含有一个元素的树，之后不断进行合并两个值最小的树，直到合并成一颗树。霍夫曼编码长度为相应字母所在树高乘频次，通过递归操作来计算编码长度。

3. 测试结果（测试输入，测试输出）

A ： 堆的操作：



B：霍夫曼编码



4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

结果与预期相同

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

A ： 堆的操作

```
#include<iostream>

template<typename T>
class MinHeap{//最小堆类
public:
    MinHeap(){//最小堆的初始化函数，完成堆的初始化操作
        int n;
        std::cin>>n;
        heap = new T[6000];//因为第一个 root 点在 1 处
        for(int i = 1; i <= n; i++){
            std::cin>>heap[i];//先记录到数组中
        }
        heapSize = n;
        for(int root = heapSize / 2; root >= 1; root --){//最后一个点是 heapSize, 最后一个含有孩子的节点是 heapSize / 2
            T rootElement = heap[root];//记录该节点元素值
            int child = 2 * root;//左孩子索引
            while(child <= heapSize){
                if(child < heapSize && heap[child] > heap[child + 1]) child ++;//右孩子存在且右孩子值小于左孩子值，则 child++
                if(rootElement <= heap[child]) break;//如果根节点 <= 最大子节点，则 break;
                heap[child / 2] = heap[child];
```

```
                child = child * 2;
            }
            //当结束时,heap[child] > rootElement, 此时 heap[child / 2]已经上移， 所以 heap[child / 2]空出，令 heap[child / 2] = rootElement;
            heap[child/2] = rootElement;
        }
    }
    ~MinHeap(){
        delete heap;
    }
    void push(T cnt){//插入操作
        heapSize++;
        int cntNodeloc = heapSize;
        while(cntNodeloc!=1 && heap[cntNodeloc / 2] > cnt){//没有到堆顶且其父节点值比 cnt 大，则父节点下沉, cntNodeloc 更新为父节点索引
            heap[cntNodeloc] = heap[cntNodeloc / 2];
            cntNodeloc = cntNodeloc / 2;
        }
        heap[cntNodeloc] = cnt;
    }
    void pop(){//弹出删除操作
        T lastElement =    heap[heapSize];
        heapSize --;
        int cntNodeloc = 1; //记录父索引 ,
        int child = 2;//child 存储较大的孩子的索引
        while(child <= heapSize){//child 索引没有到堆底时
            if(child    < heapSize && heap[child] > heap[child + 1]) child++;//找到左右孩子中较小者
            if(lastElement <= heap[child]) break;
            heap[cntNodeloc] = heap[child];
            cntNodeloc = child;
            child = 2 * child;
        }
        heap[cntNodeloc] = lastElement;
    }
    void Outheap(){
        for(int i = 1; i <= heapSize; i++){
            std::cout<<heap[i]<<' ';
        }
    }
    void heapOrder(){
        while(heapSize > 0 ){
            std::cout<<this->top()<<' ';
            this->pop();
        }
    }
```

```
    T& top(){
        return heap[1];
    }


private:
    T *heap;
    int heapSize;
};
int main(){
    MinHeap<int> heap;
    std::cout<<heap.top()<<'\n';
    int m; std::cin>>m;
    for(int i = 1; i <= m; i++){
        int sym; std::cin>>sym;
        if(sym == 1){
            int num; std::cin>>num;
            heap.push(num);
            std::cout<<heap.top()<<'\n';
        }
        else if(sym == 2){
            heap.pop();
            std::cout<<heap.top()<<'\n';
        }
        else if(sym == 3){
            MinHeap<int> Orderheap;
            Orderheap.heapOrder();
        }
    }
    return 0 ;
}
```

B：霍夫曼编码

```
#include<iostream>
struct Node{
    int key = 0;
    Node* leftptr = nullptr;
    Node* rightptr = nullptr;

};
class MinHeap{//使用最小堆
public:
    MinHeap(){
```

```
heapSize = 0;
heap = new Node*[100];
for(int i = 0; i < 100; i++){
    heap[i] = nullptr;
}
}
~MinHeap(){//析构函数
    for(int i = 0; i < 100; i++){
        delete heap[i];
    }
    delete []heap;
}
void push(Node* cnt){
    heapSize++;
    int cntNodeloc = heapSize;
    while(cntNodeloc!=1 && heap[cntNodeloc / 2]->key > cnt->key){
        heap[cntNodeloc] = heap[cntNodeloc / 2];
        cntNodeloc = cntNodeloc / 2;
    }
    heap[cntNodeloc] = cnt;
}
void pop(){
    Node* lastElement =    heap[heapSize];
    heapSize --;
    int cntNodeloc = 1;
    int child = 2;//
    while(child <= heapSize){
        if(child    < heapSize && heap[child]->key > heap[child + 1]->key) child++;
        if(lastElement->key <= heap[child]->key) break;
        heap[cntNodeloc] = heap[child];
        cntNodeloc = child;
        child = 2 * child;
    }
    heap[cntNodeloc] = lastElement;
}
void Outheap(){
    for(int i = 1; i <= heapSize; i++){
        std::cout<<heap[i]->key<<' ';
    }
}
void heapOrder(){
    while(heapSize > 0 ){
        std::cout<<this->top()<<' ';
        this->pop();
    }
}
```

```
    Node* & top(){
        return heap[1];
    }


private:
    Node **heap;//使用一个二维数组，能够让最小堆能够存储树结构
    int heapSize;
};

class HuffmanTree{
public:
    HuffmanTree(){
        heap = new MinHeap ;
        char c = getchar();
        element = new int[26];
        for(int i = 0; i < 26; i++){
            element[i] = 0;
        }
        while(c != '\n'){//输入字符串
            element[c-'a']++;
            c = getchar();
        }
        for(int i = 0; i < 26; i++){//初始把含有一个元素的树压入堆中
            if(element[i] == 0) continue;
            Node* temp = new Node;
            temp->key = element[i];
            temp->leftptr = nullptr;
            temp->rightptr = nullptr;
            heap->push(temp);
        }

    }
    ~HuffmanTree(){
        delete heap;
    }

    void merge(){//合并两个值最小的树
        Node* a,*b;
        for(int i = 1; i <= 25; i++){
            if(element[i] == 0) continue;
            //取出两个值最小的树并弹出
            a = heap->top();
            heap->pop();
            b = heap->top();
            heap->pop();
```

```
                Node* temp = new Node;
                //生成新树
                temp->key = a->key + b->key;
                temp->leftptr =    a;
                temp->rightptr = b;
                heap->push(temp);
            }
        }
        int    outans(Node* temp, int height = 0){//霍夫曼编码长度  = height *  频次
            if(temp->leftptr==nullptr && temp->rightptr == nullptr) return temp->key * height;
             else if(temp->leftptr != nullptr && temp->rightptr != nullptr) return outans(temp->leftptr,
height + 1) + outans(temp->rightptr, height + 1);
            else if(temp->leftptr != nullptr && temp->rightptr == nullptr) return outans(temp->leftptr,
height + 1);
            else    if(temp->leftptr    ==    nullptr    &&    temp->rightptr    !=    nullptr)    return
outans(temp->rightptr, height + 1);
        }
        void Outans(){
            int ans = outans(heap->top(), 0);
            std::cout<<ans;
        }
        void Outtree(Node* temp){
            std::cout<<temp->key<<'\n';
        }
private:
        MinHeap* heap;
        int * element;
};

int main(){
        HuffmanTree tree;
        tree.merge();//将每个合并
        tree.Outans();//输出霍夫曼编码长度
        return 0 ;
}
```